1.颜色的显示

开发板的分辨率：1024*600

```
每一行有1024个像素点(可以显示颜色的点)，一共有600行
从0开始数
```

三基色：红 绿 蓝（每一个分量都是一个字节)

```
r       g       b
0xff    0x00    0x00    深红色0xff0000
0x00    0xff    0x00    深绿色0xff00
```

颜色用四个字节表示(a透明度rgb)unsigned int

RGB颜色查询对照表

因为兼容性问题，色阶板功能只能在IE浏览器中运行

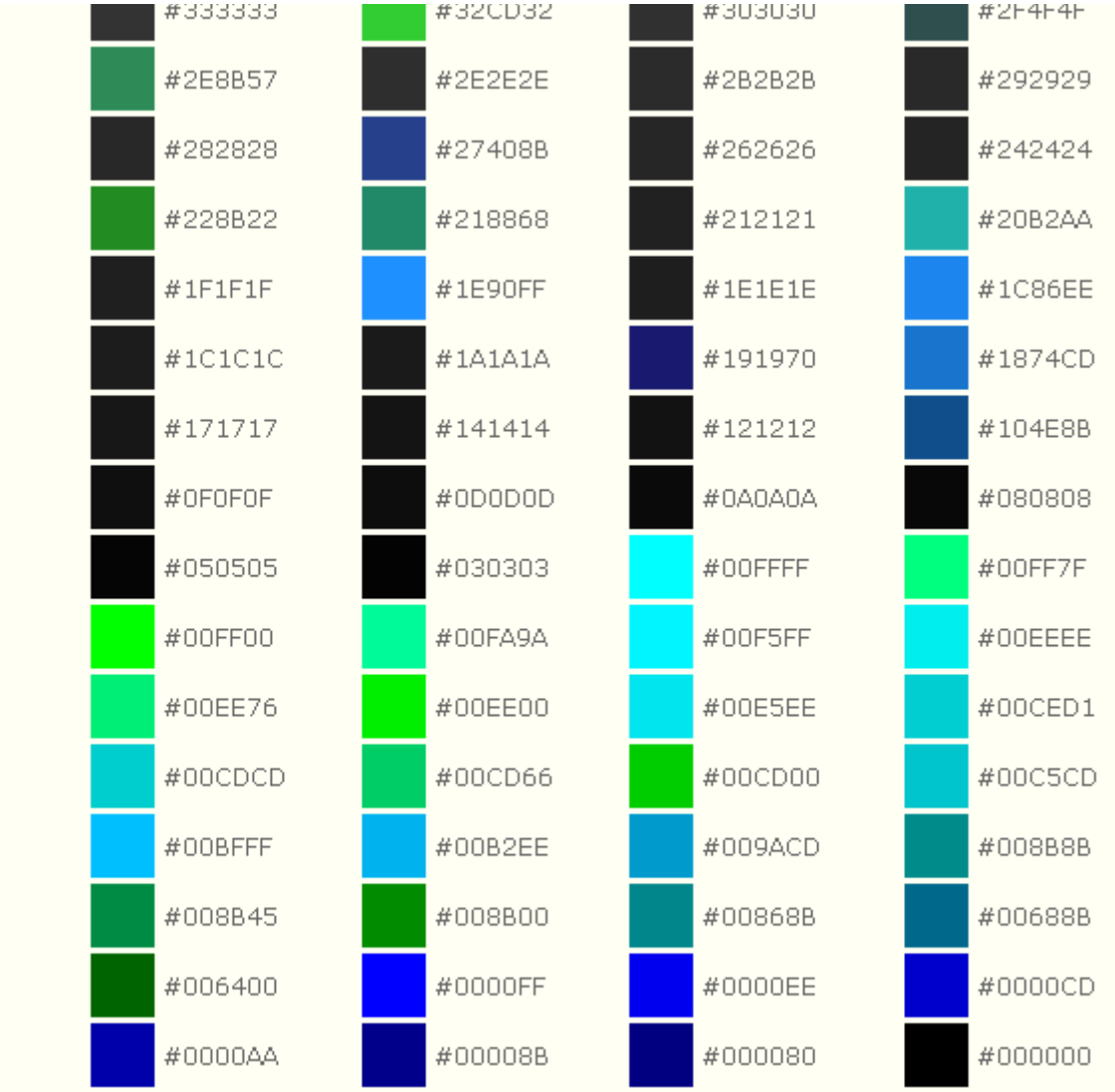
RGB颜色对照表

#FFFFFF	#FFFFF0	#FFFFE0	#FFFF00
#FFFAFA	#FFFAF0	#FFFACD	#FFF8DC
#FFF68F	#FFF5EE	#FFF0F5	#FFEFDB
#FFEFD5	#FFEC8B	#FFEBCD	#FFE7BA
#FFE4E1	#FFE4C4	#FFE4B5	#FFE1FF
#FFDEAD	#FFDAB9	#FFD700	#FFD39B
#FFC1C1	#FFC125	#FFC0CB	#FFBBFF
#FFB90F	#FFB6C1	#FFB5C5	#FFAEB9
#FFA54F	#FFA500	#FFA07A	#FF8C69
#FF8C00	#FF83FA	#FF82AB	#FF8247
#FF7F50	#FF7F24	#FF7F00	#FF7256
#FF6EB4	#FF6A6A	#FF69B4	#FF6347
#FF4500	#FF4040	#FF3E96	#FF34B3
#FF3030	#FF1493	#FF00FF	#FF0000
#FDF5E6	#FCFCFC	#FAFAFA	#FAFAD2
#FAF0E6	#FAEBD7	#FA8072	#F8F8FF
#F7F7F7	#F5FFFA	#F5F5F5	#F5F5DC
#F5DEB3	#F4F4F4	#F4A460	#F2F2F2
#F0FFFF	#F0FFF0	#F0F8FF	#F0F0F0
#F0E68C	#F08080	#EEEEE0	#EEEED1
#EEEE00	#EEE9E9	#EEE9BF	#EEE8CD
#EEE8AA	#EEE685	#EEE5DE	#EEE0E5
#EEDFCC	#EEDC82	#EED8AE	#EED5D2
#EED5B7	#EED2EE	#EECFA1	#EECBAD
#EEC900	#EEC591	#EEB4B4	#EEB422
#EEAEEE	#EEAD0E	#EEA9B8	#EEA2AD

#EE9A49	#EE9A00	#EE9572	#EE82EE
#EE8262	#EE7AE9	#EE799F	#EE7942
#EE7621	#EE7600	#EE6AA7	#EE6A50
#EE6363	#EE5C42	#EE4000	#EE3B3B
#EE3A8C	#EE30A7	#EE2C2C	#EE1289
#EE00EE	#EE0000	#EDEDED	#EBEBEB
#EAEAEA	#E9967A	#E8E8E8	#E6E6FA
#E5E5E5	#E3E3E3	#E0FFFF	#E0EEEE
#E0EEE0	#E0E0E0	#E066FF	#DEDEDE
#DEB887	#DDA0DD	#DCDCDC	#DC143C
#DBDBDB	#DB7093	#DAA520	#DA70D6
#D9D9D9	#D8BFD8	#D6D6D6	#D4D4D4
#D3D3D3	#D2B48C	#D2691E	#D1EEEE
#D1D1D1	#D15FEE	#D02090	#CFCFCF
#CDCDC1	#CDCDB4	#CDCD00	#CDC9C9
#CDC9A5	#CDC8B1	#CDC673	#CDC5BF
#CDC1C5	#CDC0B0	#CDBE70	#CDBA96
#CDB7B5	#CDB79E	#CDB5CD	#CDB38B
#CDAF95	#CDAD00	#CDAA7D	#CD9B9B
#CD9B1D	#CD96CD	#CD950C	#CD919E
#CD8C95	#CD853F	#CD8500	#CD8162
#CD7054	#CD69C9	#CD6889	#CD6839
#CD661D	#CD6600	#CD6090	#CD5C5C
#CD5B45	#CD5555	#CD4F39	#CD3700
#CD3333	#CD3278	#CD2990	#CD2626
#CD1076	#CD00CD	#CD0000	#CCCCCC
#CAFF70	#CAE1FF	#C9C9C9	#C7C7C7
#C71585	#C6E2FF	#C67171	#C5C1AA
#C4C4C4	#C2C2C2	#C1FFC1	#C1CDCD
#C1CDC1	#C1C1C1	#C0FF3E	#BFEFFF

#BFBFBF	#BF3EFF	#BEBEBE	#BDBDBD
#BDB76B	#BCEE68	#BCD2EE	#BC8F8F
#BBFFFF	#BABABA	#BA55D3	#B9D3EE
#B8B8B8	#B8860B	#B7B7B7	#B5B5B5
#B4EEB4	#B4CDCD	#B452CD	#B3EE3A
#B3B3B3	#B2DFEE	#B23AEE	#B22222
#B0E2FF	#B0E0E6	#B0C4DE	#B0B0B0
#B03060	#AEEEEE	#ADFF2F	#ADD8E6
#ADADAD	#ABABAB	#AB82FF	#AAAAAA
#A9A9A9	#A8A8A8	#A6A6A6	#A52A2A
#A4D3EE	#A3A3A3	#A2CD5A	#A2B5CD
#A1A1A1	#A0522D	#A020F0	#9FB6CD
#9F79EE	#9E9E9E	#9C9C9C	#9BCD9B
#9B30FF	#9AFF9A	#9ACD32	#9AC0CD
#9A32CD	#999999	#9932CC	#98FB98
#98F5FF	#97FFFF	#96CDCD	#969696
#949494	#9400D3	#9370DB	#919191
#912CEE	#90EE90	#8FBC8F	#8F8F8F
#8EE5EE	#8E8E8E	#8E8E38	#8E388E
#8DEEEE	#8DB6CD	#8C8C8C	#8B8B83
#8B8B7A	#8B8B00	#8B8989	#8B8970
#8B8878	#8B8682	#8B864E	#8B8386
#8B8378	#8B814C	#8B7E66	#8B7D7B
#8B7D6B	#8B7B8B	#8B795E	#8B7765
#8B7500	#8B7355	#8B6969	#8B6914
#8B668B	#8B6508	#8B636C	#8B5F65
#8B5A2B	#8B5A00	#8B5742	#8B4C39
#8B4789	#8B475D	#8B4726	#8B4513
#8B4500	#8B3E2F	#8B3A62	#8B3A3A
#8B3626	#8B2500	#8B2323	#8B2252

#8B1C62	#8B1A1A	#8B0A50	#8B008B
#8B0000	#8A8A8A	#8A2BE2	#8968CD
#87CEFF	#87CEFA	#87CEEB	#878787
#858585	#848484	#8470FF	#838B8B
#838B83	#836FFF	#828282	#7FFFD4
#7FFF00	#7F7F7F	#7EC0EE	#7D9EC0
#7D7D7D	#7D26CD	#7CFC00	#7CCD7C
#7B68EE	#7AC5CD	#7A8B8B	#7A7A7A
#7A67EE	#7A378B	#79CDCD	#787878
#778899	#76EEC6	#76EE00	#757575
#737373	#71C671	#7171C6	#708090
#707070	#6E8B3D	#6E7B8B	#6E6E6E
#6CA6CD	#6C7B8B	#6B8E23	#6B6B6B
#6A5ACD	#698B69	#698B22	#696969
#6959CD	#68838B	#68228B	#66CDAA
#66CD00	#668B8B	#666666	#6495ED
#63B8FF	#636363	#616161	#607B8B
#5F9EA0	#5E5E5E	#5D478B	#5CACEE
#5C5C5C	#5B5B5B	#595959	#575757
#556B2F	#555555	#551A8B	#54FF9F
#548B54	#545454	#53868B	#528B8B
#525252	#515151	#4F94CD	#4F4F4F
#4EEE94	#4D4D4D	#4B0082	#4A708B
#4A4A4A	#48D1CC	#4876FF	#483D8B
#474747	#473C8B	#4682B4	#458B74
#458B00	#454545	#43CD80	#436EEE
#424242	#4169E1	#40E0D0	#404040
#3D3D3D	#3CB371	#3B3B3B	#3A5FCD
#388E8E	#383838	#36648B	#363636



颜色显示的步骤

1.打开屏幕 "/dev/fb0"（开发板的路径）

```
int lcd_fd = open();
```

2.映射

```
y=2*x+3;
把x当成是一块缓冲区(内存)，间接改变y的值,把y当成是设备
建立了映射关系，操作缓冲区就像操作设备一样的

mmap映射
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
int fd, off_t offset);
addr:一般填NULL，想让系统给我们分配映射的地址，而不是自己指定
length:映射的内存空间需要多大
    1024*600*4
prot：映射过后的权限 PROT_EXEC|PROT_READ|PROT_WRITE
        PROT_EXEC  可执行
        PROT_READ  可读
        PROT_WRITE 可写
flags：映射过后的属性
    MAP_SHARED：共享属性，对内存的操作，立即再设备上显示
```

```
fd:内存要跟哪个文件建立映射关系
    lcd_fd
offset:偏移量
    0
返回值：
    成功：返回与设备建立映射关系的内存地址
    失败：NULL
```

3.显示颜色

```
给第一个点赋值红色
*plcd = 0xff0000;
给第二个点赋值红色
*(plcd+1) = 0xff0000;
给任意一个(x,y)赋值红色 x:列 y:行
*(plcd+1024*y+x) = 0xff0000;

/*
    画点子函数
    i:x轴
    j:y轴
*/
void lcd_draw_point(int i,int j,unsigned int color)
{
    if(i>=0&&i<1024 && j>=0&&j<600)
        *(plcd+1024*j+i) = color;
}

for(int i = 0;i<600;i++)//行 y轴
{
    for(int j = 0;j<1024;j++)//列 x轴
    {
        //画点函数
        lcd_draw_point(j,i,0xff0000);
    }
}
```

4.解除映射

```
int munmap(void *addr, size_t length);
参数:
    addr:释放刚才映射的地址
        plcd
    length:要释放刚才映射空间的大小
        1024*600*4
```

5.关闭屏幕

```
close(lcd_fd);
```

练习:

```
1.把上面的第一步和第二步封装成一个函数lcd_open
    函数里面的lcd_fd和plcd记得定义成全局变量
2.把上面的第四步和第五步封装一个函数lcd_close

把整个屏幕显示颜色
int main()
{
    lcd_open();
    //全部显示红色
    lcd_close();
}
写完了之后最好自己写一个lcd.c lcd.h
```

2.bmp图片显示

bmp:位图文件，是一种无压缩的图片文件格式

怎么去获取一张bmp图片?

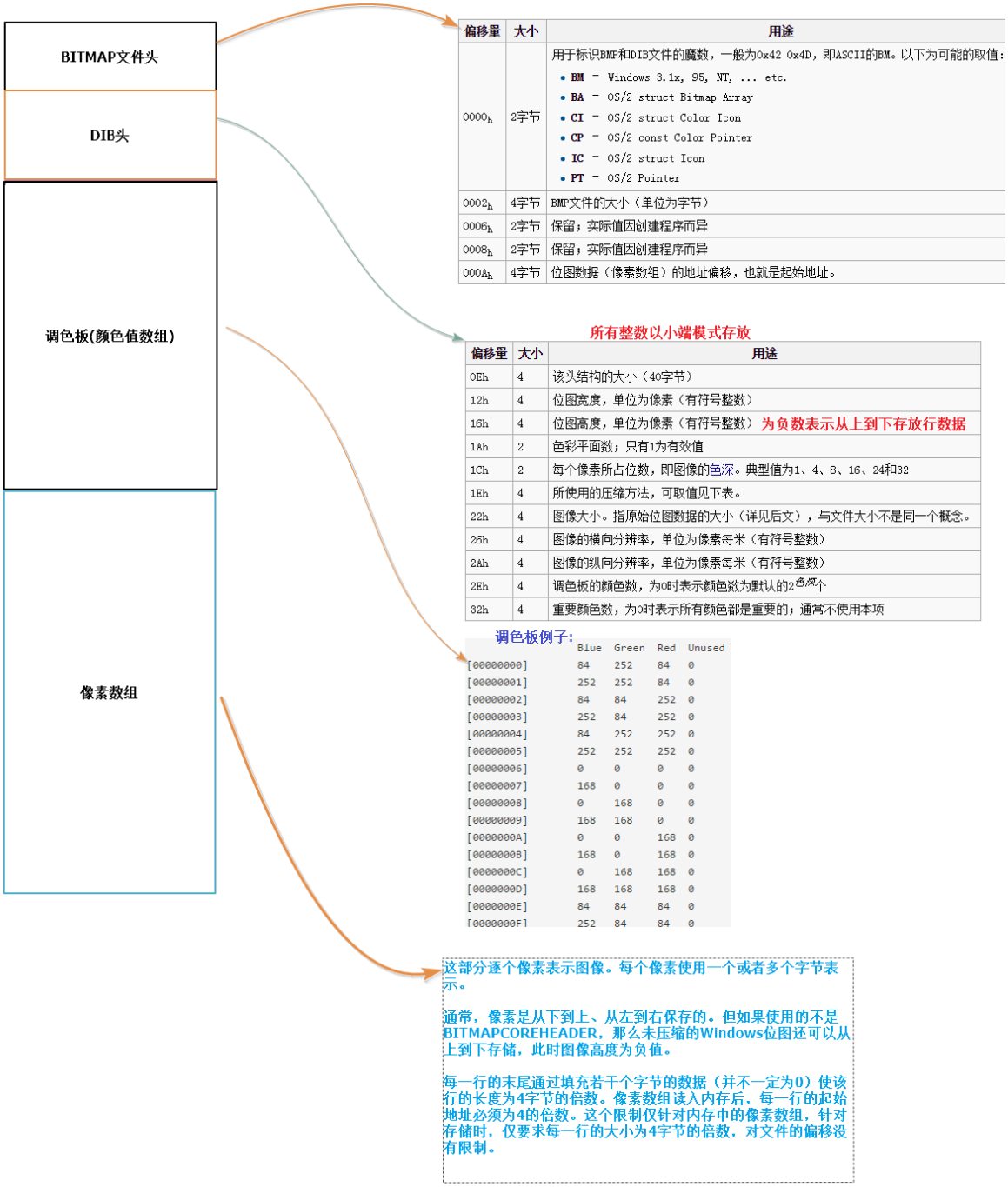
1.用画图软件打开，修改一下宽高

2.另存为bmp图片(24位图)

1.打开图片

```
int bmp_fd = open("1.bmp",O_RDWR);
```

2.读取图片



1）BITMAP文件头 14个字节

2）DIB头 40个字节

获取图片的宽 高 色深

```
int width = 0;
lseek(bmp_fd,0x12,SEEK_SET);
read(bmp_fd,&width,4);
读到的width可能是一个负数
    width > 0
        从左到右保存每一个像素点
    width < 0
        从右到左保存每一个像素点
int height = 0;
lseek(bmp_fd,0x16,SEEK_SET);
read(bmp_fd,&height,4);
读到的height可能是一个负数
    height > 0
```

```
        从下到上保存每一个像素点
    height < 0
        从上到下保存每一个像素点
int depth = 0;
lseek(bmp_fd,0x1C,SEEK_SET);
read(bmp_fd,&depth,2);
depth == 24
    rgb
depth == 32
    argb
```

3) 调色板

```
对于24位图和32位图没有调色板
```

4)像素数组

计算像素数组的大小total_bytes

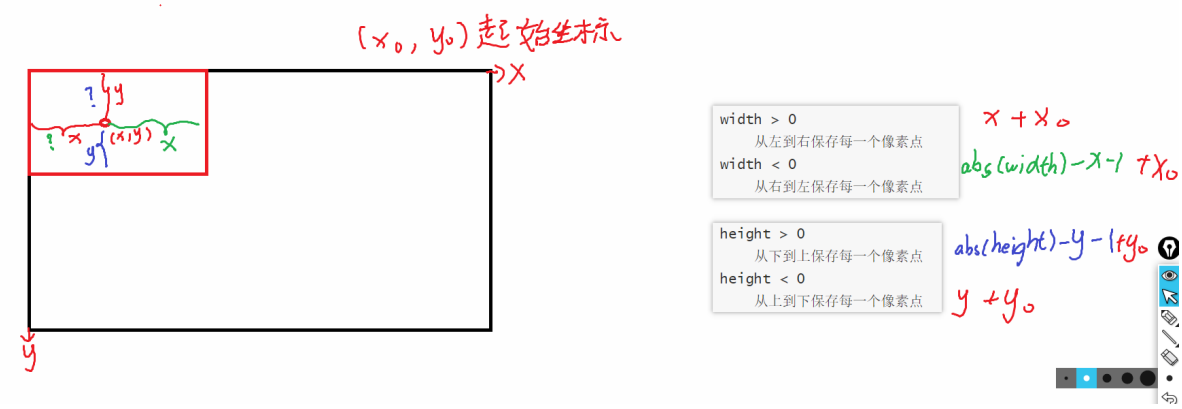
```
一行像素点的个数
    abs(width)  //abs取绝对值
每个像素点占多少个字节
    depth/8
一行有多少个字节
    int line_bytes = abs(width)*depth/8
为了保证每一行字节数都是4的倍数，就需要在末尾添加几个空白的字节
    "癞子"
每一行需要的癞子数量
    int laizi = 0;
    if(line_bytes%4)
    {
        laizi = 4-line_bytes%4;
    }
一行的总的的字节数
    line_bytes += laizi;
整个像素数组的大小
    int total_bytes = line_bytes * abs(height);
```

```
char pix[total_bytes] = {0};//用来保存读到的像素数组的数据
lseek(bmp_fd,54,SEEK_SET);//跳转光标到像素数组的位置 54
read(bmp_fd,pix, total_bytes);//读取数据放到pix中
```

3.解析像素数组数据

```
unsigned char a,r,g,b;
unsigned int color;
int x0 = 0,y0 = 0;//显示图片起始位置坐标
int i = 0;
for(int y = 0;y<abs(height);y++)//行
{
    for(int x = 0;x<abs(width);x++)//列
    {
        b = pix[i++];
        g = pix[i++];
        r = pix[i++];
```

```
        if(depth == 32)
            a = pix[i++];
        else if(depth == 24)
            a = 0;
        //组装一个颜色
        color = a<<24|r<<16|g<<8|b;
        //画点
        lcd_draw_point(width>0?x+x0:abs(width)-x-1+x0,height>0?abs(height)-y-1+y0?y+y0,color);//x(列) y(行) 颜色
    }
    //跳过癞子
    i+=laizi;
}
```



4）关闭图片

练习：把显示图片的代码进行封装

```
/*
    bmp图片显示
    x0,y0:你要显示图片的起始位置 x0列 y0行
    bmp_path：你要显示的图片名字
*/
void bmp_display(int x0,int y0,char *bmp_path)
{
    //1.打开图片

    //2.获取图片的宽高色深

    //3.获取像素数组的大小

    //4.跳转光标到像素数组的位置

    //5.读取像素数组的数据

    //6.解析像素数组的数据

    //7.关闭图片
}

int main()
{
    //lcd_open
```

```
    //bmp_display(0,0,"1.bmp");

    //lcd_close
}
```